

e-Hotels

Course Project

By:

Aidan Nikolaus, 300309124

Andrius Avenido, 300344515

Part A: DBMS and Programming Languages

Database Management System

- **PostgreSQL** — used as the relational DBMS. All tables, views, indexes, and triggers are written in standard SQL with PostgreSQL-specific extensions (e.g., SERIAL, asyncpg driver, plpgsql trigger functions). The schema is namespaced under the hotelapp schema.

Backend

- **Python 3** — primary language for the server-side application.
- **FastAPI** — web framework for building the REST API.
- **Uvicorn** — ASGI server used to run the FastAPI application.
- **asyncpg** — asynchronous PostgreSQL driver for all database queries.
- **Pydantic** — used for request/response data validation and modelling.
- **python-dotenv** — used to load environment variables (database connection string) from a .env file.

Frontend

- **TypeScript** — primary language for the client-side application.
- **React 19** — UI component library.
- **Vite** — build tool and local development server.
- **React Router 7** — client-side routing between pages.
- **Tailwind CSS** — utility-first CSS framework for styling.

Part B: Installation Guide

Prerequisites

Ensure the following are installed on your machine before proceeding:

- **PostgreSQL** (version 13 or later)
- **Python 3.10+** and pip
- **Node.js 18+** and npm

Step 1 — Set Up the Database

1. Open **pgAdmin** or a psql terminal connected to your PostgreSQL server.
2. Create a new database and schema:

```
CREATE DATABASE hoteldb;
\c hoteldb
CREATE SCHEMA hotelapp;
SET search_path TO hotelapp;
```
3. Run the SQL files in the following order (found in the SQL Codes/ folder):
 - a. createWHOLEDB.sql — creates all core tables
 - b. CreateViews.sql — creates the two database views
 - c. CreateIndexes.sql — creates performance indexes
 - d. manager trigger.sql — creates the single-manager-per-hotel trigger
 - e. preventbookingactiverental.sql — creates the booking conflict prevention trigger

Step 2 — Configure and Run the Backend

1. Navigate to the backend directory:
`cd App/Backend_Hotel`
2. (Recommended) Create and activate a Python virtual environment:

```
python -m venv venv
```

Windows:

```
venv\Scripts\activate
```

macOS/Linux:

```
source venv/bin/activate
```
3. Install dependencies:
`pip install -r requirements.txt`
4. Create a .env file in App/Backend_Hotel/ with your PostgreSQL connection string:

```
DATABASE_URL=postgresql://YOUR_USER:YOUR_PASSWORD@localhost:54
```

32/hoteldb

5. Start the backend server: `uvicorn FastAPI_Main:app --reload`
The API will be available at <http://localhost:8000>.

Step 3 — Configure and Run the Frontend

1. Navigate to the frontend directory: `cd App/Frontend_Hotel`
2. Install dependencies: `npm install`
3. Start the development server: `npm run dev`
The application will be available at <http://localhost:5173>.

Part C: DDL Statements

Schema

`SET search_path TO hotelapp;`

Core Tables

```
CREATE TABLE Hotel_Chain (  
  chain_id SERIAL PRIMARY KEY,  
  street VARCHAR(255),  
  city VARCHAR(100),  
  zip_code VARCHAR(20),  
  country VARCHAR(100),  
  chain_name VARCHAR(100)  
);
```

```
CREATE TABLE Employee (  
  employee_id SERIAL PRIMARY KEY,  
  ssn_sin VARCHAR(255) UNIQUE NOT NULL,  
  full_name VARCHAR(255) NOT NULL,  
  street VARCHAR(255) NOT NULL,  
  city VARCHAR(255) NOT NULL,
```

```
zip_code VARCHAR(255),
country VARCHAR(255) NOT NULL,
hotel_id INTEGER,
FOREIGN KEY (hotel_id) REFERENCES Hotel(hotel_id)
);
```

```
CREATE TABLE Customer (
customer_id SERIAL PRIMARY KEY,
phone_number VARCHAR(255) UNIQUE NOT NULL,
full_name VARCHAR(255) NOT NULL,
street VARCHAR(255) NOT NULL,
city VARCHAR(255) NOT NULL,
zip_code VARCHAR(255) NOT NULL,
country VARCHAR(255) NOT NULL,
ssn_sin VARCHAR(255) NOT NULL,
id_type VARCHAR(255) NOT NULL,
date_of_registration VARCHAR(255) NOT NULL
);
```

Hotel

```
CREATE TABLE Hotel (
hotel_id SERIAL PRIMARY KEY,
chain_id INTEGER,
rating INTEGER,
street VARCHAR(255),
city VARCHAR(100),
zip_code VARCHAR(20),
country VARCHAR(100),
email_address VARCHAR(255) NOT NULL,
manager_id INTEGER,
FOREIGN KEY (chain_id) REFERENCES Hotel_Chain(chain_id),
FOREIGN KEY (manager_id) REFERENCES Employee(employee_id)
);
```

Contact Information

```
CREATE TABLE Chain_Numbers (  
    chain_id INTEGER,  
    phone_number VARCHAR(20) NOT NULL,  
    PRIMARY KEY (chain_id, phone_number),  
    FOREIGN KEY (chain_id) REFERENCES Hotel_Chain(chain_id)  
);
```

```
CREATE TABLE Chain_Email (  
    chain_id INTEGER,  
    email_address VARCHAR(255) NOT NULL,  
    PRIMARY KEY (chain_id, email_address),  
    FOREIGN KEY (chain_id) REFERENCES Hotel_Chain(chain_id)  
);
```

```
CREATE TABLE Hotel_Numbers (  
    hotel_id INTEGER,  
    phone_number VARCHAR(20) NOT NULL,  
    PRIMARY KEY (hotel_id, phone_number),  
    FOREIGN KEY (hotel_id) REFERENCES Hotel(hotel_id)  
);
```

Rooms

```
CREATE TABLE Room (  
    hotel_id INTEGER,  
    room_number VARCHAR(4),  
    price INTEGER,  
    amenities VARCHAR(255),  
    capacity INTEGER,  
    room_view VARCHAR(255),  
    bed_extension BOOLEAN,  
    PRIMARY KEY (hotel_id, room_number),  
    FOREIGN KEY (hotel_id) REFERENCES Hotel(hotel_id)  
);
```

```
CREATE TABLE Room_Damages (  
    hotel_id INTEGER,  
    room_number VARCHAR(4),  
    damage_type VARCHAR(255) NOT NULL,  
    PRIMARY KEY (hotel_id, room_number, damage_type),  
    FOREIGN KEY (hotel_id, room_number) REFERENCES Room(hotel_id, room_number)  
);
```

Bookings & Renting

```
CREATE TABLE Booking (  
    booking_id SERIAL PRIMARY KEY,  
    customer_id INTEGER,  
    customer_name VARCHAR NOT NULL,  
    room_number VARCHAR(4) NOT NULL,  
    hotel_chain VARCHAR NOT NULL,  
    hotel VARCHAR NOT NULL,  
    hotel_id INTEGER NOT NULL,  
    price DECIMAL NOT NULL CHECK (price > 0),  
    status VARCHAR(20) NOT NULL,  
    FOREIGN KEY (customer_id) REFERENCES Customer(customer_id),  
    FOREIGN KEY (hotel_id, room_number) REFERENCES Room(hotel_id, room_number),  
    CHECK (status IN ('Booked', 'Cancelled', 'Completed'))  
);
```

```
CREATE TABLE Renting (  
    renting_id SERIAL PRIMARY KEY,  
    customer_id INTEGER,  
    customer_name VARCHAR NOT NULL,  
    hotel VARCHAR NOT NULL,  
    hotel_chain VARCHAR NOT NULL,  
    room_number VARCHAR(4) NOT NULL,  
    hotel_id INTEGER NOT NULL,  
    price DECIMAL NOT NULL CHECK (price > 0),
```

```
check_in_date VARCHAR NOT NULL,  
check_out_date VARCHAR,  
FOREIGN KEY (customer_id) REFERENCES Customer(customer_id),  
FOREIGN KEY (hotel_id, room_number) REFERENCES Room(hotel_id, room_number)  
);
```

```
CREATE TABLE Works_On (  
    employee_id INTEGER,  
    renting_id INTEGER,  
    last_modified TIMESTAMP,  
    PRIMARY KEY (employee_id, renting_id),  
    FOREIGN KEY (employee_id) REFERENCES Employee(employee_id),  
    FOREIGN KEY (renting_id) REFERENCES Renting(renting_id)  
);
```

```
CREATE TABLE Employee_Positions (  
    employee_id INTEGER,  
    role_positions VARCHAR(100) NOT NULL,  
    PRIMARY KEY (employee_id, role_positions),  
    FOREIGN KEY (employee_id) REFERENCES Employee(employee_id)  
);
```

Archive Tables

```
CREATE TABLE Archive_Booking (  
    booking_id SERIAL PRIMARY KEY,  
    customer_id INTEGER,  
    customer_name VARCHAR NOT NULL,  
    room_number VARCHAR(4) NOT NULL,  
    hotel_chain VARCHAR NOT NULL,  
    hotel VARCHAR NOT NULL,  
    hotel_id INTEGER NOT NULL,  
    price DECIMAL NOT NULL CHECK (price > 0),  
    status VARCHAR(20) NOT NULL,  
    CHECK (status IN ('Booked', 'Cancelled', 'Completed'))  
);
```



```
CREATE TABLE Archive_Renting (  
    renting_id SERIAL PRIMARY KEY,  
    customer_id INTEGER,  
    customer_name VARCHAR NOT NULL,  
    hotel VARCHAR NOT NULL,  
    hotel_chain VARCHAR NOT NULL,  
    room_number VARCHAR(4) NOT NULL,  
    hotel_id INTEGER NOT NULL,  
    price DECIMAL NOT NULL CHECK (price > 0),  
    check_in_date VARCHAR NOT NULL,  
    check_out_date VARCHAR  
);
```

Views

```
CREATE VIEW available_rooms_per_city AS  
SELECT  
    h.city,  
    COUNT(r.room_number) AS available_rooms  
FROM Room r  
JOIN Hotel h ON r.hotel_id = h.hotel_id  
LEFT JOIN Renting rent  
    ON r.hotel_id = rent.hotel_id AND r.room_number = rent.room_number  
    AND (rent.check_out_date IS NULL OR rent.check_out_date::DATE >= CURRENT_DATE)  
WHERE rent.renting_id IS NULL  
GROUP BY h.city  
ORDER BY h.city;
```

```
CREATE OR REPLACE VIEW hotel_total_capacity AS  
SELECT  
    h.hotel_id,  
    hc.chain_name,  
    h.email_address,  
    SUM(r.capacity) AS total_capacity  
FROM Hotel h
```

```
JOIN Hotel_Chain hc ON h.chain_id = hc.chain_id
JOIN Room r ON h.hotel_id = r.hotel_id
GROUP BY h.hotel_id, hc.chain_name, h.email_address
ORDER BY h.hotel_id;
```

Indexes

```
-- Speeds up booking lookups by customer
CREATE INDEX idx_booking_customer ON Booking(customer_id);

-- Speeds up booking lookups by hotel
CREATE INDEX idx_booking_hotel ON Booking(hotel_id);

-- Speeds up room lookups by hotel (used when customers browse rooms)
CREATE INDEX idx_room_hotel ON Room(hotel_id);

-- Speeds up employee work assignment lookups
CREATE INDEX idx_workson_employee ON Works_On(employee_id);

-- Speeds up hotel filtering by chain
CREATE INDEX idx_hotel_chain ON Hotel(chain_id);
```

Triggers

```
-- Prevents a hotel from having more than one manager
CREATE OR REPLACE FUNCTION check_single_manager()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.role_positions = 'Manager' THEN
        IF EXISTS (
            SELECT 1
            FROM Employee_Positions ep
            JOIN Employee e ON ep.employee_id = e.employee_id
            WHERE ep.role_positions = 'Manager'
```

```

        AND e.hotel_id = (
            SELECT hotel_id FROM hotelapp.Employee WHERE employee_id =
NEW.employee_id
        )
    ) THEN
        RAISE EXCEPTION 'This hotel already has a manager.';
    END IF;
END IF;
RETURN NEW;
END;
$$ LANGUAGE plpgsql;

```

```

CREATE TRIGGER one_manager_per_hotel
BEFORE INSERT ON hotelapp.Employee_Positions
FOR EACH ROW
EXECUTE FUNCTION check_single_manager();

```

-- Prevents booking a room that is currently actively rented

```

CREATE OR REPLACE FUNCTION prevent_booking_if_rented()
RETURNS TRIGGER AS $$

```

```

BEGIN

```

```

    IF EXISTS (
        SELECT 1
        FROM Renting r
        WHERE r.room_number = NEW.room_number
            AND r.hotel = NEW.hotel
            AND r.hotel_chain = NEW.hotel_chain
            AND r.hotel_id = NEW.hotel_id
            AND r.check_out_date IS NULL
    ) THEN

```

```

    ) THEN
        RAISE EXCEPTION 'Cannot create booking: room is currently rented or overlaps with an
active renting';

```

```

    END IF;
    RETURN NEW;

```

```

END;
$$ LANGUAGE plpgsql;

```

```
CREATE TRIGGER before_booking_insert  
BEFORE INSERT ON Booking  
FOR EACH ROW  
EXECUTE FUNCTION prevent_booking_if_rented();
```